Complete Project Documentation: Talent Bridge Careers

1. Project Overview

- **Project Name:** Talent Bridge Careers

- **Project Description:** An intelligent, AI-powered Applicant Tracking System (ATS) that connects job seekers with recruiters while automating the initial screening process.

- **Problem Statement:** Recruiters spend countless hours manually reading hundreds of resumes to find suitable candidates. Traditional ATS systems rely on basic keyword matching, which often misses context and rejects good candidates.

- **Proposed Solution:** A platform that utilizes Large Language Models (LLMs) to contextually read candidate resumes, compare them against the specific job description, and generate an intelligent Match Score and summary instantly.

- **Project Objectives:** To reduce resume screening time by 90%, improve candidate quality, and provide a seamless application experience.

- **Target Users:**
    - **Candidates:** Looking for jobs and tracking their application status.
    - **Admins/Recruiters:** Managing job postings and evaluating applicants.

- **Key Benefits:** Non-blocking UI, instant AI evaluations, automated email communications, and secure role-based access.

- **Major Features:** Custom Email JWT Authentication, PDF Resume parsing, Asynchronous AI processing, Automated Status Emails.

- **Project Scope:** Core ATS functionalities from user registration to application rejection/shortlisting.

- **Project Limitations**: Currently uses local file storage and basic Python threading instead of enterprise task queues.

2. Technology Stack

Category	Technology	Purpose
Frontend	React (Vite/CRA)	User Interface and client-side routing.
Backend	Django & DRF	REST API, Business Logic, and ORM.
Database	SQLite / MySQL	Relational data storage for users and applications.
AI / LLM	Google Gemini	Reading resumes and generating AI Match Scores.
Authentication	SimpleJWT	Secure, stateless email-based login.
PDF Extraction	PyPDF2	Extracting raw text from uploaded PDF resumes.
Email	Django SMTP	Automated status notifications and alerts.

Styling	Tailwind CSS	Rapid, responsive UI design.

Why did we choose this technology?

- **Django/React:** Provides a robust, decoupled architecture where the backend focuses heavily on data logic while React delivers a fast, dynamic user experience.
- **Gemini 3.6 Flash:** Offers an incredibly fast response time, massive context window (ideal for multi-page resumes), and native JSON output formatting, making it perfect for structured ATS data.

3. System Requirements

Software:

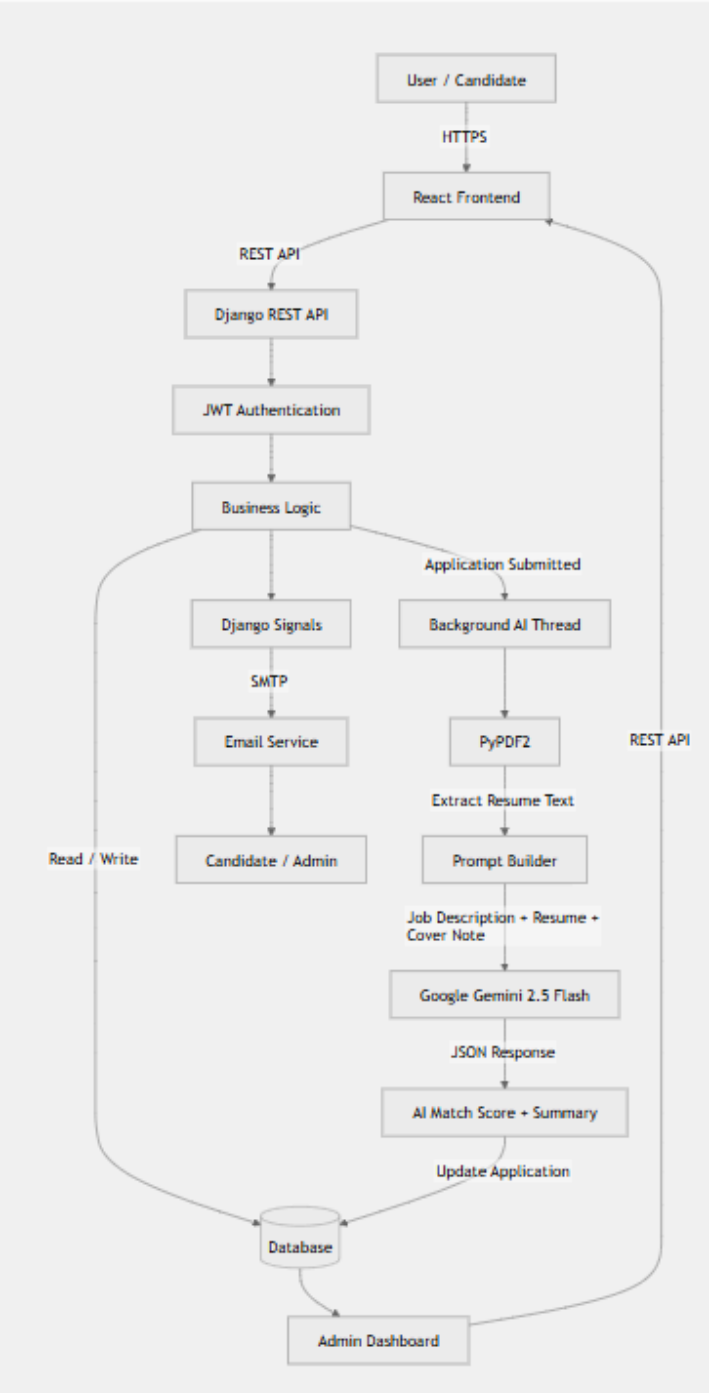
- Python: v3.10+
- Node.js: v18+
- npm: v9+
- Git
- Database: SQLite (Default) or MySQL
- Browser: Chrome, Firefox, Safari, or Edge
- IDE: VS Code (Recommended)

Hardware:

- Minimum RAM: 4GB (8GB Recommended)
- Storage: 500MB free space
- Processor: Dual-core 2.0 GHz or higher

4. System Architecture

The application follows a decoupled client-server architecture. The frontend communicates with the backend exclusively via RESTful APIs.


User / Candidate
HTTPS
React Frontend
REST API
Django REST API
JWT Authentication
Business Logic
Application Submitted
Django Signals
Background AI Thread
SMTP
Email Service
PyPDF2
REST API
Extract Resume Text
Read / Write
Candidate / Admin
Prompt Builder
Job Description + Resume + Cover Note
Google Gemini 2.5 Flash
JSON Response
AI Match Score + Summary
Update Application
Database
Admin Dashboard

5. Project Folder Structure

frontend: Contains all UI elements, state management, and API calling logic (Axios).
backend: Contains the database models, authentication logic, background threads, and external API integrations.

6. User Roles

1. Candidate

- Register an account.
- Login using Email and Password.
- View public job listings.
- Apply to jobs (Upload bio-data and PDF resume).
- View their application status on the Candidate Dashboard.

2. Admin (Recruiter)

- Login using Admin credentials.
- View all submitted applications.
- View AI Match Scores and AI Summaries.
- Change application status (New, Reviewed, Shortlisted, Rejected).
- Delete rejected applications.

7. Complete Feature Documentation

Candidate Registration

- Purpose: Allow users to create an account.
- Input: Full Name, Email, Password.
- Validation: Checks password length and unique email constraints.
- Backend Process: Creates a CustomUser with role='CANDIDATE'.
- Database: Inserts into CustomUser table.
- Response: 201 Created.
- Error Handling: Displays Django validation errors via SweetAlert.

Job Application Submission

- Purpose: Connect a candidate to a specific job opening.
- User Flow: Fills out bio-data form -> Uploads PDF -> Submits.
- API: POST /api/jobs/<id>/apply/
- Database: Creates an Application record.
- AI Processing: Instantly triggers a background thread to analyze the resume via Gemini, preventing the UI from freezing.

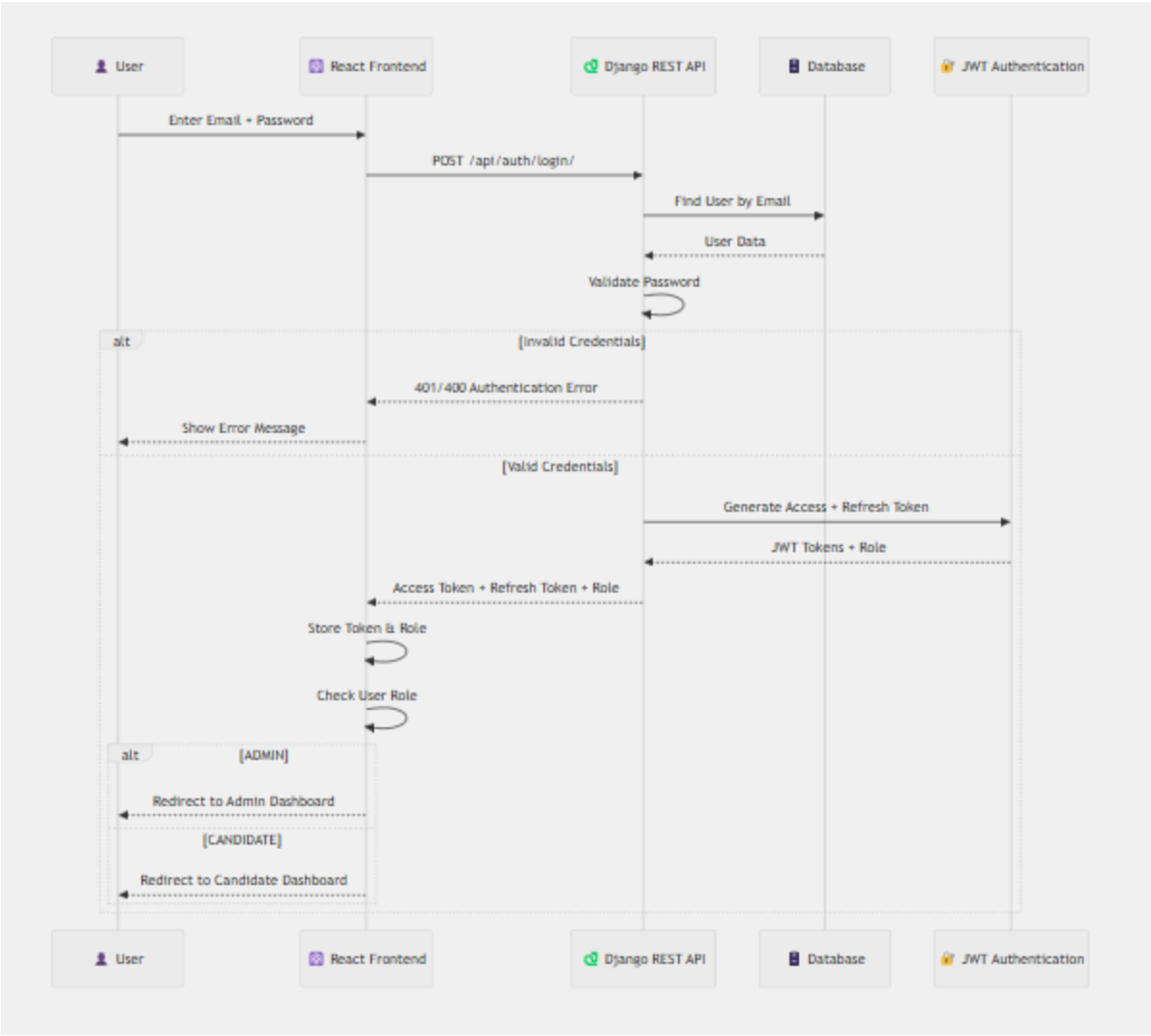
Admin Status Update

- Purpose: Move candidates through the hiring pipeline.
- User Flow: Admin selects a new status from a dropdown in the dashboard.
- Backend Process: Updates status. If status is REJECTED, a Django Signal (post_save) detects the change and fires off a polite rejection email.

8. Authentication & Authorization

The system uses a custom JSON Web Token (JWT) approach that allows users to log in with an Email instead of a Username.

- **Registration:** Passwords are hashed using Django's PBKDF2 algorithm before saving.
- **Login:** CustomTokenObtainPairSerializer intercepts the email, authenticates the user, and injects the user's role into the JWT payload.
- **Protected Routes:** React uses <ProtectedRoute> and <PublicRoute> wrappers to ensure Admins cannot access Candidate pages, and unauthenticated users are forced to log in.


User
React Frontend
Django REST API
Database
JWT Authentication
Enter Email + Password
POST /api/auth/login/
Find User by Email
User Data
Validate Password
alt
[Invalid Credentials]
401/400 Authentication Error
Show Error Message
[Valid Credentials]
Generate Access + Refresh Token
JWT Tokens + Role
Access Token + Refresh Token + Role
Store Token & Role
Check User Role
alt
[ADMIN]
Redirect to Admin Dashboard
[CANDIDATE]
Redirect to Candidate Dashboard
User
React Frontend
Django REST API
Database
JWT Authentication

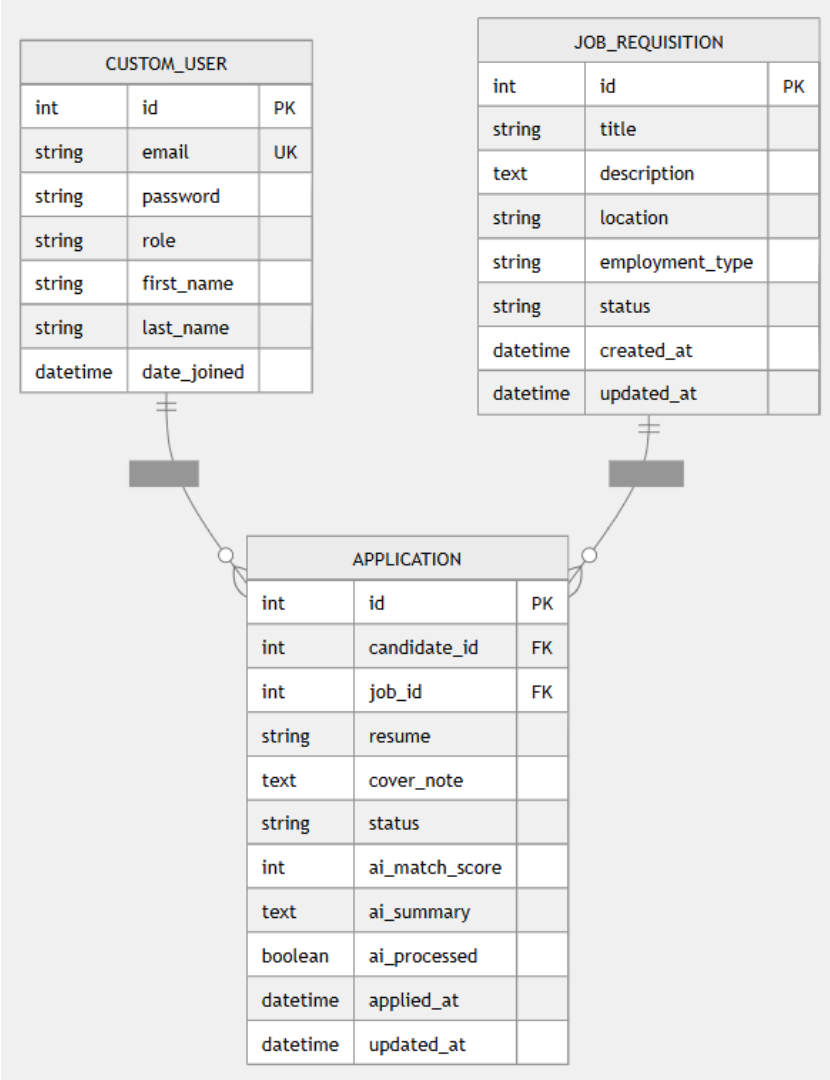
9. Database Documentation

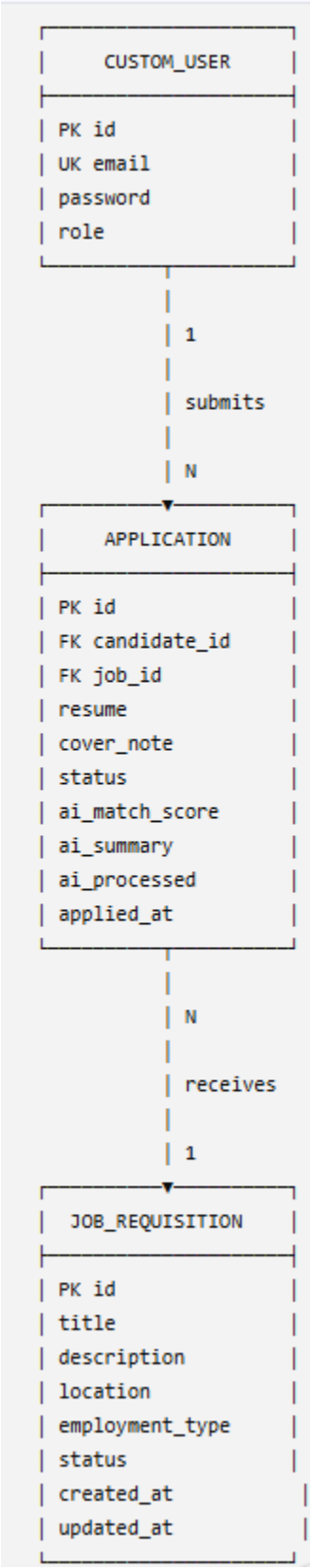
- Database Technology: SQLite (Development) / MySQL (Production)

Models

1. CustomUser: Extends AbstractUser. Uses email as the primary identifier. Includes a role field.
2. JobRequisition: Stores job title, description, and requirements.
3. Application: Links a Candidate to a Job. Stores bio-data, the resume file path, and AI evaluation metrics.

ER Diagram




CUSTOM_USER
PK id
UK email
password
role
1
submits
N
APPLICATION
PK id
FK candidate_id
FK job_id
resume
cover_note
status
ai_match_score
ai_summary
ai_processed
applied_at
N
receives
1
JOB_REQUISITION
PK id
title
description
location
employment_type
status
created_at
updated_at

10. API Documentation

Method	Endpoint	Auth	Purpose
POST	/api/auth/register/	No	Registers a new Candidate
POST	/api/auth/login/	No	Authenticates user, returns JWT & role
POST	/api/jobs/<id>/apply/	Yes	Submits application, triggers AI thread
GET	/api/admin/applications/	Admin	Fetches applications with AI data
PATCH	/api/admin/applications/<id>/status/	Admin	Updates status (triggers emails

11. AI/LLM Integration

- **AI Provider:** Google Gemini
- **Model:** `gemini-3.6-flash`
- **SDK**: google-genai (Latest official SDK)
- **Purpose:** To cross-reference a candidate's uploaded resume with the Job Description and provide a quantitative match score.
- **Input:** Job Description text + Extracted Resume text + Cover Note text.

Processing Flow:

1. Resume PDF uploaded.
2. PyPDF2 reads all pages and extracts raw text.
3. Text is injected into a strict prompt.
4. Gemini processes the prompt using forced JSON output formatting.
5. Django parses the JSON and saves ai_match_score and ai_summary.

12. AI Prompt Architecture

- Prompt Location: Hardcoded in backend/core/ai_utils.py.
- Variables: {job_description}, {resume_text}, {cover_note}.
- Expected Response: The model config forces application/json MIME type.
- Error Handling: Wrapped in a try/except block. If the API fails, the application saves a score of 0 and a summary stating "AI Analysis failed."

Prompt Example:

You are an expert technical AI recruiter. Evaluate the candidate against the Job Description.

Job Description: {job_description}

Candidate Resume: {resume_text}

Candidate Cover Note: {cover_note}

Analyze the match based on required skills, experience, and domain knowledge.

Output strictly in JSON format matching this structure:

{{

"score": <integer from 0 to 100>,

"summary": "<A 3-bullet-point summary explaining the score>"

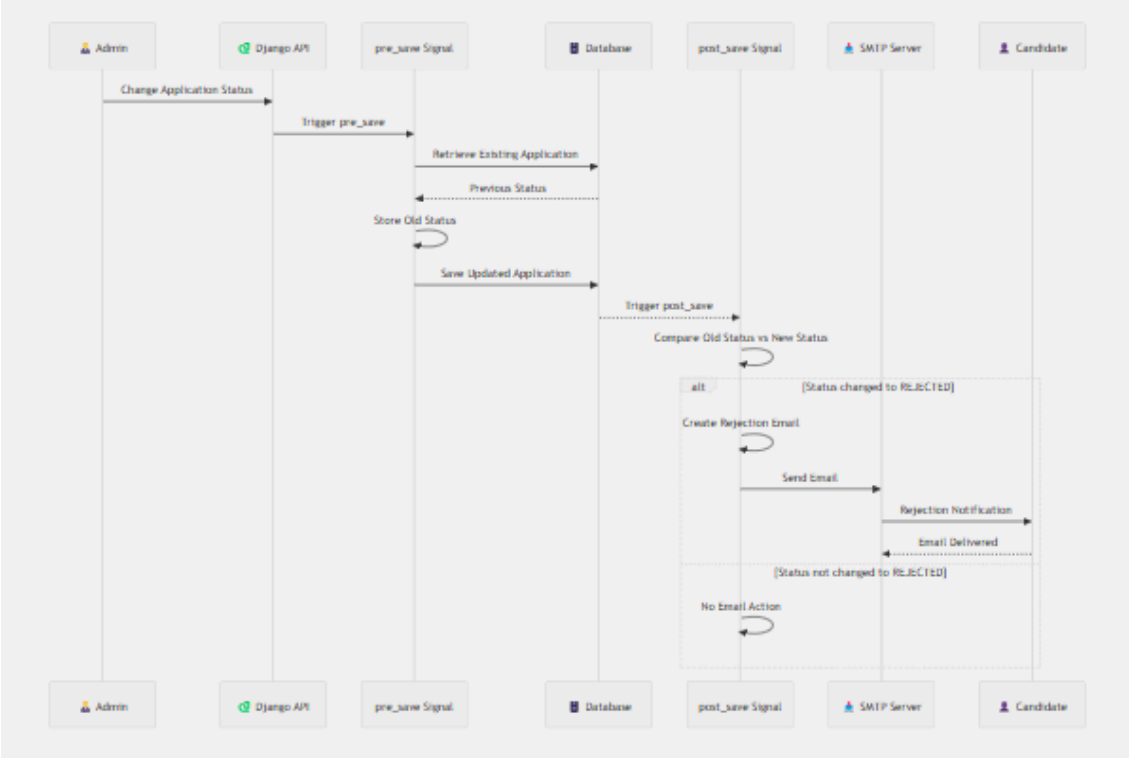
}}

13. AI Processing / Background Task

- Implementation: Python's built-in threading.Thread.
- Why? API calls to Gemini and PDF extraction take 3-5 seconds. If done synchronously, the candidate's browser would freeze while waiting for the AI. Threading allows Django to instantly return a "Success" message to the user while the AI works invisibly.
- Limitation: Python threads share memory and do not persist if the server crashes. For production, Celery + Redis is highly recommended as a reliable task queue.

14. Email System

- Email Provider: Django SMTP (Configurable to Gmail, SendGrid, Amazon SES).
- Email Triggers (via Django Signals):
    - *Candidate Notification:* Sent instantly when an application is saved.
    - *Admin Alert:* Sent to System Admin when an application arrives.
    - *Rejection Email:* Sent when post_save detects status has changed specifically to REJECTED.
- Error Handling: fail_silently=True is utilized so that if the SMTP server is down, the application logic does not crash the app.



15. Setting configuration for apis and emails

Create the database if you use mysql

Run this command in mysql workbench

CREATE DATABASE `candidate_sourcing_db;`

SECRET_KEY=your_django_secret_key

```
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.mysql',
        'NAME': 'candidate_sourcing_db',
        'USER': '',          # Your MySQL username
        'PASSWORD': '', # Your MySQL password
        'HOST': '127.0.0.1',
        'PORT': '3306',
    }
}


AUTH_USER_MODEL = 'core.CustomUser'


# --- Real Email Configuration (Gmail SMTP) ---
EMAIL_BACKEND = 'django.core.mail.backends.smtp.EmailBackend'
EMAIL_HOST = 'smtp.gmail.com'
EMAIL_PORT = 587
EMAIL_USE_TLS = True

# The Gmail address you are using to send the emails
EMAIL_HOST_USER = 'email'

# The 16-character App Password (NO SPACES)
EMAIL_HOST_PASSWORD = 'app password'

# Who the emails appear to come from (usually the same as
EMAIL_HOST_USER)
DEFAULT_FROM_EMAIL = 'email'
```

```
# Where you want the "New Application Received" alerts to be sent
SYSTEM_ADMIN_EMAIL = 'email'

# JWT Token Configuration
SIMPLE_JWT = {
    'ACCESS_TOKEN_LIFETIME': timedelta(days=7),
    'REFRESH_TOKEN_LIFETIME': timedelta(days=14),
    'ROTATE_REFRESH_TOKENS': False,
    'BLACKLIST_AFTER_ROTATION': False,
    'UPDATE_LAST_LOGIN': False,
}



#LLM Configuration api key
GEMINI_API_KEY = ""
```

FRONTEND_URL=http://localhost:5173

16. API Key Configuration

How to get the Gemini API Key:

1. Go to Google AI Studio.
2. Log in with your Google account.
3. Click "Get API Key" on the left menu.
4. Click "Create API key in new project".
5. Copy the generated key (starts with AIza...) and paste it into your .env file as GEMINI_API_KEY.

17. Installation Guide

Step 1: Clone the repository

Bash
git clone <your-repo-url>
cd project-root

Step 2: Setup Backend Environment

Create the database

Run this command in mysql workbench

CREATE DATABASE `candidate_sourcing_db;`

Bash
cd backend
python -m venv venv
source venv/bin/activate # Windows: venv\Scripts\activate

Step 3: Install Backend Dependencies

Bash
pip install -r requirements.txt

Step 4: Setup Frontend Environment

npm install react-router-dom axios sweetalert2 react-toastify

npm install -D tailwindcss@3 postcss autoprefixer

npx tailwindcss init -p

This installs Tailwind CSS 3.x and creates:

tailwind.config.js

postcss.config.js

1. Configure `tailwind.config.js`

```
/** @type {import('tailwindcss').Config} */

export default {

  content: [

    "./index.html",

    "./src/**/*.{js,ts,jsx,tsx}",

  ],

  theme: {

    extend: {},

  },

  plugins: [],

}
```

2. Add Tailwind to your CSS

In `src/index.css`:

```
@tailwind base;

@tailwind components;

@tailwind utilities;
```

Bash
cd frontend
npm install

Step 5: Database Migration & Admin Creation

Bash
cd backend

```
python manage.py makemigrations
python manage.py migrate
python manage.py createsuperuser  # Set role to ADMIN if requested
```

18. Configuration Guide

- CORS: In settings.py, ensure CORS_ALLOWED_ORIGINS includes your frontend URL (e.g., http://localhost:5173).
- Media Files: Ensure MEDIA_URL and MEDIA_ROOT are configured in settings so uploaded PDFs are saved properly.

19. Running the Project

You will need two separate terminal windows.

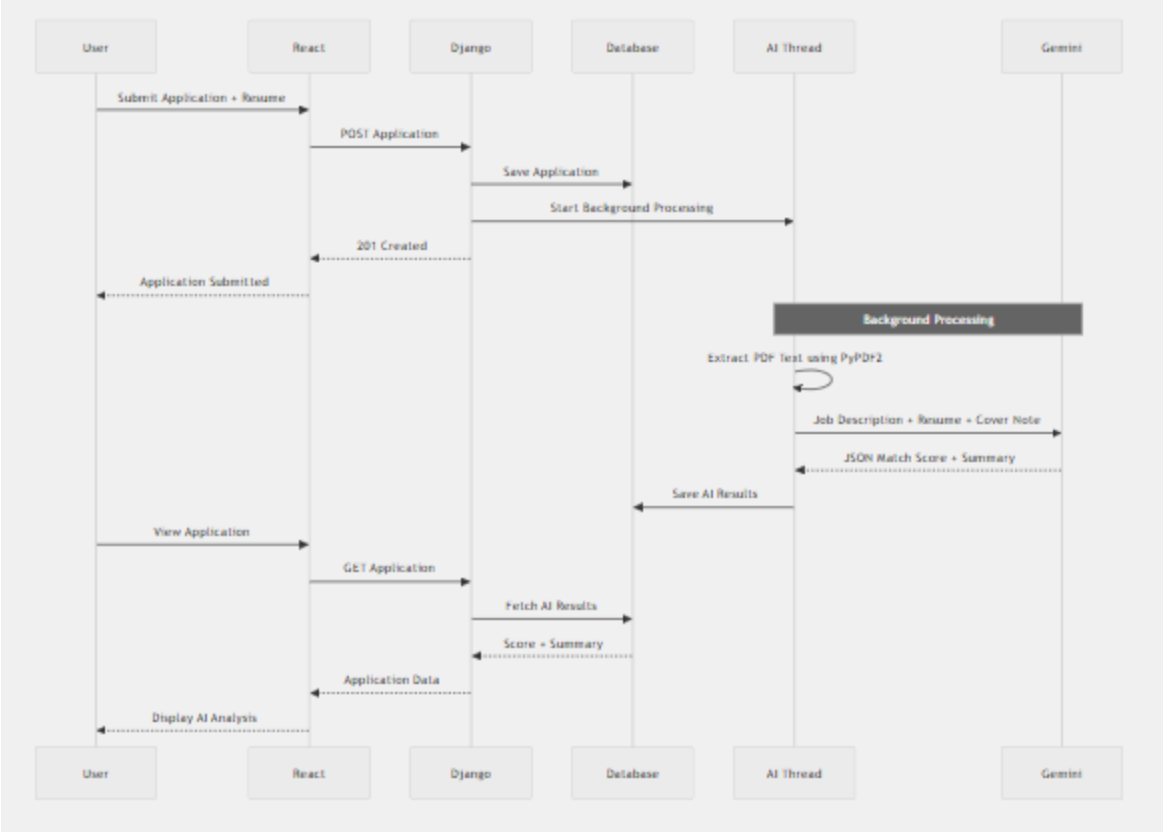
Terminal 1 (Backend):

Bash
```
cd backend
source venv/bin/activate or venv\Scripts\activate
python manage.py runserver
```

Terminal 2 (Frontend):

Bash
```
cd frontend
npm run dev
```

Open your browser to the local URL provided by Vite (usually http://localhost:5173).



20. Security

Currently Implemented:

- Password Hashing: Django's default PBKDF2 hashing.
- JWT Security: Stateless tokens prevent session hijacking.
- Role-based Access: React <ProtectedRoute> prevents candidates from viewing admin pages. Django IsAdminUser prevents API tampering.
- File Upload Validation: Models restricted to specific file types (PDF, DOCX).
- Fail-Silently: SMTP errors do not expose server stack traces to users.

21. Known Limitations & Future Improvements

Current Limitations:

- **Threading vs Task Queue:** Currently utilizes Python's built-in `threading.Thread` for AI processing. If the server crashes during processing, the thread is lost.
- **Storage:** Uses local file storage for Resume PDFs.

Future Improvements:

- **Celery + Redis:** Integrate a robust task queue for asynchronous AI processing and email dispatching to ensure zero data loss.
- **Cloud Storage:** Migrate media file storage to AWS S3.
- **Advanced AI Features:** Implement Vector Databases and RAG (Retrieval-Augmented Generation) to allow semantic search across all candidate resumes.